RAPPORT SYNTHÉTIQUE

TP1 – Application CRUD d'inscription des étudiants

Objectif :

Le projet est une application web minimale qui réalise les quatre opérations CRUD (créer, lire, modifier, supprimer) sur une table d'étudiants. Elle est écrite en PHP avec PDO sur MySQL, sans framework ni bibliothèque externe : la feuille de style est locale et l'application fonctionne sans Internet. Elle s'exécute sous XAMPP en copiant le dossier dans htdocs.

Le code est court (environ 13 Ko, six fichiers) et fait de chaque fichier une responsabilité unique, ce qui le rend facile à lire et à expliquer.

Organisation du projet :

Fichier	Opération	Rôle
config.php		Connexion PDO, fonction d'échappement h(), liste des champs du formulaire (CHAMPS)
index.php	Read	Liste des étudiants triée par nom, avec recherche facultative (?q=) sur le nom ou le prénom
form.php	Create / Update	Formulaire partagé : vierge pour une création, prérempli (?id=) pour une modification
save.php	Create / Update	Traitement du formulaire en POST : validation, puis INSERT ou UPDATE
delete.php	Delete	Suppression en POST après confirmation dans le navigateur
style.css	—	Feuille de style locale, palette en variables CSS (bleu, vert, rouge)

Données manipulées :

L'application lit et écrit la table etudiant de la base universite (connexion en root sans mot de passe, configuration par défaut de XAMPP). L'archive ne contient pas de script SQL : la base et la table doivent exister avant le premier lancement. Les douze champs attendus sont :

Groupe (formulaire)	Colonnes
Identité	idEtu (matricule, clé primaire), nomEtu, prenomEtu, dateNaiss
Contact	emailEtu, numTelEtu
Localisation	regEtu, departEtu, arrondisEtu
Formation	mentionEtu, cycleEtu, ParcoursEtu

Le regroupement en quatre blocs n'existe que dans l'affichage ; la liste des champs est définie une seule fois dans la constante CHAMPS, que le formulaire, la validation et les requêtes réutilisent.

Fonctionnement des quatre opérations

- **Read :** index.php exécute une requête préparée avec un paramètre nommé :q appliqué à nomEtu et prenomEtu (LIKE). Le tableau affiche matricule, nom, prénom, e-mail, mention et cycle, avec le nombre de résultats et un message « Aucun résultat » si besoin.
- **Create :** le bouton « Nouvel étudiant » ouvre form.php sans paramètre. À l'envoi, save.php construit dynamiquement l'INSERT à partir de CHAMPS.
- **Update :** form.php?id=… charge l'étudiant ; le matricule est en lecture seule car il sert de critère WHERE. Un champ caché « mode » indique à save.php s'il faut faire un UPDATE ou un INSERT.
- **Delete :** chaque ligne contient un petit formulaire POST avec confirmation JavaScript ; delete.php refuse toute méthode autre que POST (code 405).

Après chaque opération, l'utilisateur est redirigé vers la liste avec un message de succès (?ok=1) ou d'erreur (?err=…).

Sécurité et robustesse :

Mesure	Effet
PDO, requêtes préparées	Les valeurs saisies ne sont jamais concaténées dans le SQL : protection contre l'injection SQL
Fonction h() sur toutes les sorties	htmlspecialchars systématique : protection contre le XSS, y compris pour le message d'erreur de l'URL
Suppression par POST uniquement	Une simple URL ne peut pas effacer un enregistrement
Revalidation côté serveur	save.php refuse un champ vide et vérifie l'e-mail avec FILTER_VALIDATE_EMAIL, sans faire confiance au navigateur
Gestion des erreurs PDO	Mode exception ; le détail technique va dans les journaux (error_log), l'utilisateur voit un message neutre
Doublons	Le code SQLSTATE 23000 est traduit en « Matricule ou email déjà utilisé »
Noms de colonnes constants	Les colonnes viennent de CHAMPS et non de la saisie, ce qui rend l'INSERT/UPDATE dynamique sûr

Points d'attention et améliorations

- **Cohérence de la configuration :** le commentaire de config.php cite la base « universiteMbdTest », alors que le code se connecte à « universite ». Il faut créer la base sous le nom utilisé dans le code, ou corriger la ligne dsn.
- **Jeton CSRF :** les formulaires de modification et de suppression n'en ont pas. L'ajouter (jeton en session) est l'amélioration de sécurité la plus utile.
- **Compte MySQL :** root sans mot de passe convient en local seulement ; un utilisateur dédié avec droits limités serait requis en production.
- **Suppression :** delete.php ne vérifie pas que le matricule existe et annonce toujours « Opération réussie ».
- **Recherche :** le LIKE avec %…% ne peut pas utiliser un index, et les caractères % et _ saisis ne sont pas échappés. La liste n'est pas paginée : acceptable pour un TP, à compléter avec LIMIT/OFFSET sur de gros volumes.

Validation : seuls l'e-mail et la présence des champs sont contrôlés ; le téléphone et la date de naissance pourraient l'être aussi. Mise en route

1) Démarrer Apache et MySQL dans XAMPP.

2) Créer la base universite avec la table etudiant (douze colonnes ci-dessus), sans modifier le code.

3) Copier le dossier du projet dans C:\xampp\htdocs\.

4) Ouvrir http://localhost/crud-minAppliInscriptionEtu/.

Conclusion :

L'application remplit son rôle de TP : un CRUD complet, lisible et correctement protégé contre l'injection SQL et le XSS. Les principales marges de progrès sont le jeton CSRF, le compte MySQL dédié et un script SQL fourni avec le projet.